

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Industry Problem Solving Task

COURSE NAME:	Cloud Computing and Big Data Analytics	COURSE CODE:	CSA1558
---------------------	---	---------------------	---------

COURSE OUTCOME COVERED

CO4: *Analyze big data characteristics and challenges across domains including security and application aspects.*
(BL4)

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 40

Duration :60 Minutes

Code of Conduct

I **Yarra Pawan (192424294)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **Y. Pawan**

Assessment Tasks

Industry Problem Solving Task

1. **Retail Data Optimization Problem** - A retail chain operates both physical stores and an online platform. It generates large volumes of sales, inventory, and customer behavior data. The company wants real-time inventory tracking and demand forecasting. It also aims to deliver personalized marketing recommendations to customers. Design a big data architecture to support these requirements. Explain the tools, technologies, and analytics models you would use.
2. **Social Media Fraud Detection Problem** - A social media platform processes billions of user interactions daily. These include posts, comments, likes, and shares. The platform wants to detect fake accounts and misinformation quickly. Real-time or near real-time processing is required. Design a scalable big data solution for this use case. Explain the role of machine learning and stream processing frameworks.
3. **Government Data Platform Problem** A government agency is building a national data platform. It combines census, economic, and geospatial datasets. Different departments need access to shared and integrated data. The system must ensure interoperability across multiple agencies. Strict privacy and security requirements must be followed. Propose a governance and data management strategy.
4. **Banking Fraud Detection Problem**
A bank processes millions of financial transactions every minute. It needs to detect fraudulent activities in real time. The system must analyze historical and streaming transaction data. Low latency and high accuracy are critical requirements. Design a big data pipeline for fraud detection. Explain the technologies and algorithms you would implement.

5. Agriculture Smart Farming Problem

A smart farming system collects soil, weather, and crop data. Sensors and drones provide real-time field information. Farmers want insights to improve yield and resource usage. Data is generated from multiple distributed sources. Design a big data solution for precision agriculture. Explain how analytics can support decision-making.

Industry Problem Solving Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Industry Context	20%	Demonstrates strong understanding of real-world problem context	Good understanding	Basic understanding	Weak understanding
Application of Big Data Concepts	25%	Applies highly relevant big data ideas and tools	Mostly appropriate application	Partial application	Weak application
Solution Design	25%	Solution is practical, scalable, and well-reasoned	Reasonable solution	Basic solution with gaps	Unclear solution
Security / Risk Awareness	15%	Strong awareness of security and operational risks	Reasonable awareness	Limited awareness	Poor awareness
Clarity and Professionalism	15%	Well-structured and professional response	Generally clear	Moderately clear	Poorly presented

Total Marks Awarded:

Problem 1: Retail Data Optimization

Scenario

A retail chain operates both physical stores and an online platform. It generates large volumes of sales, inventory, and customer behavior data. The company wants real-time inventory tracking and demand forecasting. It also aims to deliver personalized marketing recommendations to customers. Design a big data architecture to support these requirements. Explain the tools, technologies, and analytics models you would use.

Focus Areas for This Problem

- Real-time inventory synchronization across stores and online channel
- Demand forecasting using historical + real-time sales data
- Personalized recommendations based on customer behavior

1. Problem Understanding & Industry Context

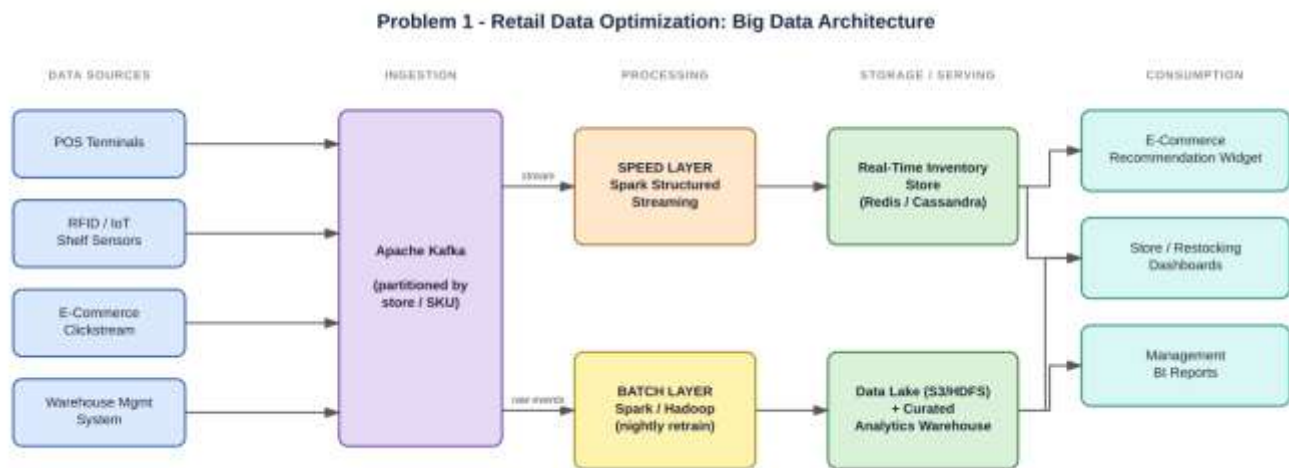
Modern retail runs on razor-thin margins where stock-outs and overstocking directly erode revenue, and the shift to omnichannel shopping means a customer's online and in-store experience must be perceived as one continuous relationship rather than two disconnected systems. This retail chain sits at the intersection of two large, high-velocity data-generating channels — physical POS/RFID systems in stores and an e-commerce platform — that must be reconciled into a single, trustworthy view of inventory and customer intent. The core industry pain point is latency: a sale rung up in-store must be reflected in the online product catalog within seconds to avoid overselling, while demand forecasting and personalization models benefit from months or years of historical trend data that only batch processing can economically handle at scale. Industry benchmarks in retail big data initiatives typically target under 5% stock-out rates and double-digit lifts in conversion from personalization, which sets the bar for how responsive and accurate this architecture needs to be.

Guiding Questions — Answered

- **What data sources feed this system, and how frequently does each update?** Point-of-sale terminals and RFID/shelf sensors emit near-instant events (seconds), the e-commerce clickstream and mobile app emit continuous session events, the warehouse/inventory management system updates on stock movement (minutes), and supplier/procurement feeds update daily. Loyalty and CRM profile data changes infrequently and is treated as slowly-changing dimension data.

- **Which parts of the pipeline need real-time processing vs. batch processing?** Inventory-count updates, cart abandonment triggers, and next-best-offer recommendations must be real-time (sub-second to a few seconds) because they change what a customer sees right now. Demand forecasting, customer segmentation, cohort analysis, and financial reporting can run as nightly or hourly batch jobs since they inform planning rather than an in-the-moment decision.
- **What forecasting or recommendation algorithm would you use, and why?** Demand forecasting uses a time-series model (Facebook Prophet for interpretable seasonality, or an LSTM where promotions/weather add non-linear effects) because retail demand is strongly seasonal and event-driven. Recommendations use ALS matrix factorization (collaborative filtering) blended with content-based features to solve the cold-start problem for new products and new customers.
- **How would you handle data from both online and offline channels consistently?** Both channels publish events to the same Kafka topics using one canonical product/customer schema (shared SKU IDs, a unified customer identity resolved via loyalty ID or hashed email), so downstream consumers never need channel-specific logic. A master data management layer reconciles duplicate customer identities across web, app, and in-store loyalty sign-ups.

2. Proposed Big Data Architecture



3. Tools & Technologies Mapping

Big Data Layer / Function	Tool(s) / Technology Proposed	Why This Tool? (reason)
Data Sources	POS terminals, RFID/IoT shelf sensors, e-commerce clickstream, mobile app events, warehouse management system	Covers every offline and online touchpoint needed to build one unified inventory and behavior view
Ingestion	Apache Kafka (partitioned by store/SKU)	High-throughput, durable, replayable event bus that decouples producers (stores/web) from consumers and preserves ordering per SKU
Storage	Amazon S3 / HDFS data lake (raw + curated zones) with Cassandra/Redis for hot lookups	Cheap, durable long-term storage for model training data plus millisecond-latency reads for live stock counts
Processing (Batch)	Apache Spark (batch) on the data lake, orchestrated by Airflow	Efficient large-scale joins/aggregations for demand forecasting, segmentation, and nightly dimension rebuilds
Processing (Real-Time)	Spark Structured Streaming / Apache Flink	Updates inventory counts and fires recommendation triggers within seconds of an event occurring
Analytics / ML	Spark MLlib; Prophet/LSTM for forecasting; ALS collaborative filtering	Purpose-built, scalable libraries for time-series

Big Data Layer / Function	Tool(s) / Technology Proposed	Why This Tool? (reason)
	+ content-based features for recommendations	forecasting and personalization
Visualization / Reporting	Power BI / Tableau dashboards for merchandising and store ops	Gives store managers and category teams an interpretable, near-real-time view of stock and demand
Security / Governance	IAM roles + Apache Ranger policies, TLS encryption in transit, tokenization for payment-linked fields	Restricts and audits who can access customer behavior and payment-adjacent data, and protects it in motion

4. Analytics / Machine Learning Models Used

Demand forecasting: a time-series model (Prophet for interpretable trend/seasonality decomposition, or an LSTM where promotions and external variables like weather add non-linear effects) is trained per SKU-store combination on historical sales blended with real-time signals, producing rolling demand forecasts that drive automated replenishment. Forecast accuracy is tracked using MAPE (mean absolute percentage error) so under-performing SKU forecasts can be flagged for manual review.

Personalized recommendations: ALS-based collaborative filtering identifies latent customer-product affinity from purchase and browsing history, blended with content-based features (category, price band, brand) to handle cold-start items and customers. Recommendations are re-scored using a lightweight real-time re-ranker that factors in current-session behavior (what the customer just viewed).

Proactive stock-out prevention: a binary classification model (gradient-boosted trees) predicts the probability of a SKU stocking out within 24-48 hours per store, using current stock velocity, promotional calendar, and forecast demand as features, triggering automated replenishment alerts before the stock-out actually happens.

5. Security, Privacy & Risk Considerations

Customer behavior and purchase history are personally identifiable and must be encrypted both at rest (AES-256 in the data lake) and in transit (TLS across Kafka and API calls), with role-based access so marketing analysts see aggregated cohort behavior rather than raw identity-linked records. Payment-linked POS data falls under PCI-DSS scope, so card data is tokenized at the point of capture and never stored in the analytics lake in raw form. A defined data-retention policy purges granular clickstream events after a set window (e.g., 18 months) while retaining only aggregated features for long-term forecasting. Because personalization uses behavioral profiling, the platform needs a clear consent/opt-out mechanism so customers can decline tracking-based recommendations in line with data-protection regulation (e.g., GDPR/DPDP-style consent requirements). Access to the real-time inventory store and forecasting pipeline is further restricted through network segmentation so store-level staff cannot query other stores' or corporate financial data.

6. Key Challenges & Mitigation Strategies

Challenge	Mitigation Strategy
Reconciling in-store and online inventory counts without double-counting or lag	Route all inventory-affecting events (sale, return, transfer) through the same Kafka topic with idempotent, exactly-once processing in the streaming layer so counts stay consistent regardless of channel
Cold-start problem for new products/customers with no purchase history	Blend collaborative filtering with content-based features (category, price, attributes) so recommendations remain reasonable before behavioral data accumulates
Forecast accuracy degrading during promotions or unusual demand spikes	Feed promotional calendar and external signals (holidays, weather) as explicit model features, and retrain forecasting models on a rolling weekly basis rather than a static schedule

Challenge	Mitigation Strategy
Scaling the real-time layer during peak shopping periods (e.g., festive sales)	Auto-scale the Kafka consumer group and Spark Streaming executors based on topic lag, and pre-provision capacity ahead of known high-traffic events

7. Conclusion / Business Value

This architecture keeps online and offline inventory synchronized within seconds, reduces stock-outs and overstocking through SKU-level predictive forecasting, and increases conversion and basket size via personalized recommendations. Because the batch and streaming layers share one ingestion backbone rather than running as two disconnected systems, the design also keeps infrastructure and reconciliation costs lower than maintaining separate online/offline data pipelines — translating directly into higher revenue, lower carrying costs, and a more consistent customer experience across channels.

Problem 2: Social Media Fraud Detection

Scenario

A social media platform processes billions of user interactions daily. These include posts, comments, likes, and shares. The platform wants to detect fake accounts and misinformation quickly. Real-time or near real-time processing is required. Design a scalable big data solution for this use case. Explain the role of machine learning and stream processing frameworks.

Focus Areas for This Problem

- Scalability to billions of daily events
- Near real-time fake account and misinformation detection
- Balancing detection accuracy with platform performance

1. Problem Understanding & Industry Context

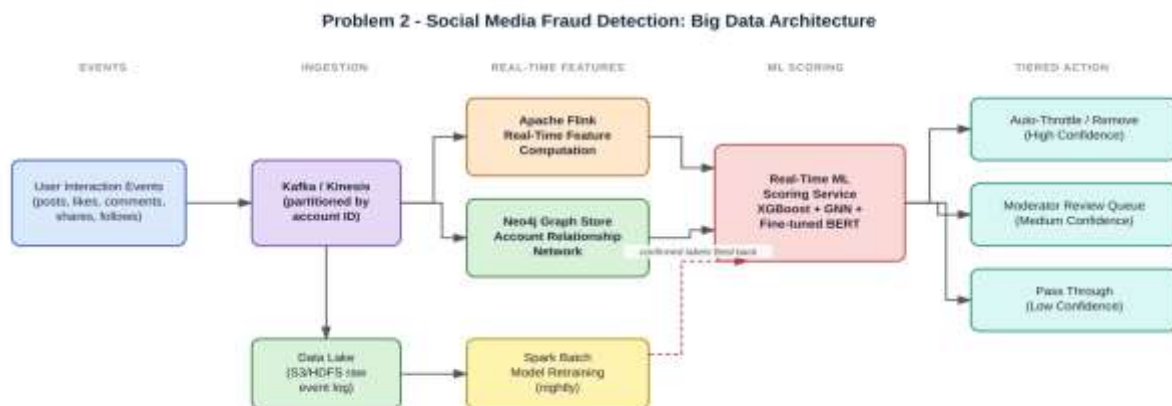
At billions-of-events-per-day scale, trust and safety is as much a systems-engineering problem as a machine learning one: the platform's reputation and advertiser confidence depend on catching coordinated fake-account networks and misinformation campaigns quickly, without silencing genuine users through overzealous automated moderation. Misinformation and fake-account networks tend to exhibit coordinated, non-organic patterns — accounts created in bursts, unnaturally synchronized posting/liking behavior, and content that is copy-pasted or minimally altered across many accounts — which distinguishes them from the noisy but organic behavior of genuine users. Because the volume here is orders of magnitude larger than the other four scenarios, the emphasis shifts from architecture sophistication to horizontal scalability and cost-efficient stream processing, while still requiring near-real-time (seconds, not hours) detection so that misinformation is caught before it goes viral.

Guiding Questions — Answered

- **What features would a machine learning model use to flag a fake account or misinformation post?** Account-graph structure (follower/following ratios, clustering with known bad actors, account age), behavioral velocity (abnormal posting/liking/following rate spikes), content features (text embeddings, duplicate/near-duplicate detection across accounts, sentiment/toxicity scores), and device/network fingerprints (shared IP ranges, emulator signatures) across many accounts registered in a short window.

- **Which stream processing framework fits billions of events/day, and why?** Apache Flink is well suited because it offers exactly-once, low-latency, horizontally scalable stateful stream processing with native support for windowed aggregations (e.g., posts-per-minute), which is exactly the kind of stateful velocity feature this use case needs at massive scale.
- **How would you reduce false positives that could wrongly flag genuine users?** Use tiered confidence scoring rather than a single automatic action: high-confidence cases are auto-throttled, medium-confidence cases go to a human moderation queue, and low-confidence cases pass through untouched. Regularly auditing the model against manually-labeled genuine-user samples also catches systematic bias before it affects real users at scale.
- **How would you scale storage and compute as user activity grows?** Partition Kafka topics and the account graph store by account ID so throughput scales horizontally by adding partitions/nodes, auto-scale the Flink cluster based on consumer lag, and keep the data lake on cheap object storage (S3/HDFS) that scales independently of the compute layer.

2. Proposed Big Data Architecture



3. Tools & Technologies Mapping

Big Data Layer / Function	Tool(s) / Technology Proposed	Why This Tool? (reason)
Data Sources	Post/comment/like/share/follow event APIs, account metadata, device/network signals	Provides the content, behavioral, and network signals needed for fraud and misinformation detection
Ingestion	Apache Kafka / Amazon Kinesis (partitioned by account ID)	Handles billions of events/day with ordered, horizontally-scalable throughput

Big Data Layer / Function	Tool(s) / Technology Proposed	Why This Tool? (reason)
Storage	HDFS/S3 data lake for raw events + Neo4j graph database for account relationships	The graph DB is purpose-built for detecting fake-account rings and coordinated networks that a flat table cannot represent well
Processing (Batch)	Apache Spark	Retrains models nightly on the full historical dataset and rebuilds graph features from confirmed moderation outcomes
Processing (Real-Time)	Apache Flink	Low-latency, exactly-once stateful stream processing suited to billions of events per day and windowed velocity features
Analytics / ML	XGBoost/Random Forest risk scoring, Graph Neural Network for account rings, fine-tuned BERT NLP classifier for content	Combines behavioral, graph, and content signals for high-precision detection with low false positives
Visualization / Reporting	Kibana/Grafana moderation dashboards	Lets trust & safety teams monitor flagged content, queue backlog, and system health in real time
Security / Governance	RBAC for moderator tools, model-bias audits, rate-limited and authenticated ingestion APIs	Prevents misuse of moderation tools and guards against biased over-flagging of specific user groups

4. Analytics / Machine Learning Models Used

Behavioral risk scoring: a gradient-boosted tree model (XGBoost) combines behavioral and metadata features (posting velocity, account age, device fingerprint reuse) into a real-time risk score, chosen for its strong performance on tabular features and fast inference latency.

Network-level detection: a graph neural network (or simpler graph-clustering algorithm such as label propagation) analyzes the follower/interaction graph to identify coordinated rings of fake accounts that behave normally individually but abnormally as a group — a pattern invisible to single-account models.

Content-level detection: a transformer-based NLP classifier (fine-tuned BERT) scores post and comment text for misinformation patterns and near-duplicate content spread across many accounts, complemented by perceptual hashing for images/video.

All three scores are combined into a single tiered confidence system, and only high-confidence combined scores trigger automatic action — directly addressing the false-positive concern from the guiding questions.

5. Security, Privacy & Risk Considerations

User interaction and behavioral data are sensitive, so access to raw graph and behavioral data is restricted to the trust & safety team via RBAC, with broader analytics or research use requiring anonymization first. Because moderation actions affect user expression and account standing, model decisions should be explainable and auditable (e.g., logging which features drove a given risk score), with periodic bias audits to ensure certain demographics or regions are not disproportionately flagged. Rate limiting and mutual authentication on ingestion and moderation APIs prevent adversaries from probing the detection system to learn how to evade it. Data retention for behavioral/graph features used purely for fraud scoring should be time-bounded, and moderator access to flagged content should itself be logged for internal accountability.

6. Key Challenges & Mitigation Strategies

Challenge	Mitigation Strategy
Adversaries adapting behavior once they learn how detection works (adversarial evasion)	Continuously retrain models on newly confirmed cases, rotate/obfuscate which exact features drive scoring, and monitor for sudden drops in detection rate as a signal of evasion
Processing billions of events/day without unacceptable latency or cost	Partition by account ID for horizontal scaling, use windowed stateful aggregation in Flink instead of recomputing from scratch, and tier storage so only recent hot data sits on fast storage
Coordinated fake-account rings that look individually normal	Layer graph-based network analysis on top of per-account scoring so coordinated group behavior is caught even when individual accounts pass single-account checks

Challenge	Mitigation Strategy
Balancing free expression with misinformation removal	Use tiered confidence routing so only very high-confidence cases are auto-actioned, with human moderator review and an appeal process for medium-confidence flags

7. Conclusion / Business Value

This design detects fake accounts and misinformation within seconds of posting while scaling horizontally to billions of daily events through partitioned, stateful stream processing. The tiered, human-in-the-loop review process keeps false positives low, protecting genuine users' experience and platform trust, while the combination of behavioral, graph, and content-level ML signals catches coordinated manipulation that any single detection approach would miss on its own.

Problem 3: Government Data Platform

Scenario

A government agency is building a national data platform. It combines census, economic, and geospatial datasets. Different departments need access to shared and integrated data. The system must ensure interoperability across multiple agencies. Strict privacy and security requirements must be followed. Propose a governance and data management strategy.

Focus Areas for This Problem

- Integration of heterogeneous datasets (census, economic, geospatial)
- Interoperability and controlled access across departments
- Data governance, privacy, and compliance

1. Problem Understanding & Industry Context

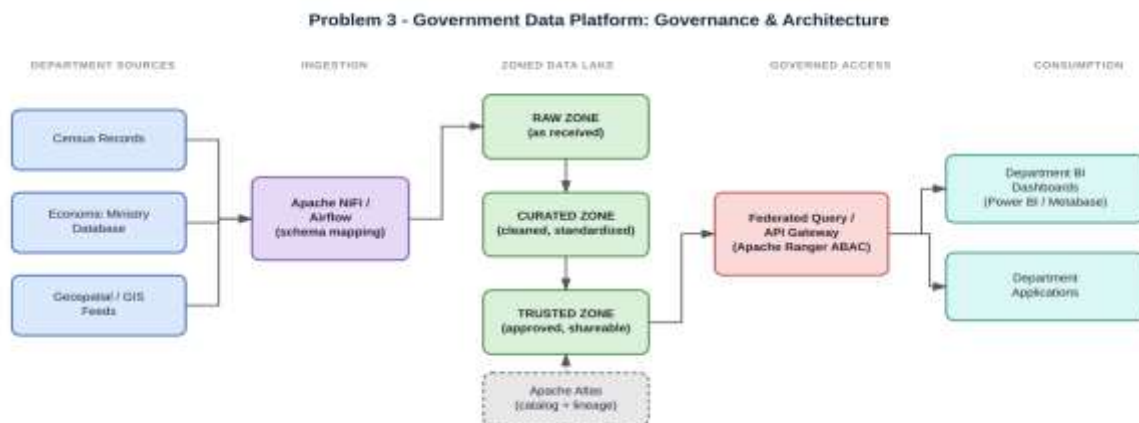
Government data platforms differ from commercial big data systems in priority order: interoperability, governance, and citizen-privacy protection come before raw processing speed. Departments typically run on legacy, heterogeneous systems — mainframes, department-specific relational databases, and standalone GIS servers — so genuine integration is impossible until data is standardized into a shared model rather than simply co-located in one storage system. Census, economic, and geospatial data also carry very different sensitivity profiles: individual census records are highly sensitive PII, aggregate economic indicators are less sensitive but commercially/politically consequential, and geospatial infrastructure data may itself be security-sensitive (e.g., critical infrastructure locations). A successful platform therefore needs both a technical integration layer and an equally strong governance layer that many purely technical big data designs underweight.

Guiding Questions — Answered

- **How would you standardize data formats across departments with different systems?** Define a canonical data model with agreed schemas for common entities (person, household, region, transaction), and map each department's legacy export format (fixed-width files, proprietary DB exports, shapefiles) into that canonical model during ingestion using Apache NiFi/Airflow. Geospatial data specifically is normalized to open OGC standards (GeoJSON, WMS/WFS) so any department's GIS tool can consume it.

- **What access control model would you use to manage who can see what data?** Attribute-based access control (ABAC), because access needs depend on multiple contextual attributes (department, role, security clearance, purpose of query, data sensitivity tier) rather than a fixed role hierarchy — a static RBAC model would be too coarse for cross-agency data sharing with varying sensitivity levels.
- **How would you anonymize or protect personally identifiable information (PII)?** Field-level tokenization/encryption for identifiable fields (national ID, name, address) in the curated zone, with decryption only for authorized, logged queries; published aggregate statistics use k-anonymity or differential privacy so individuals cannot be re-identified from released datasets, especially for small-population geographic cells.
- **What governance roles/processes (data stewards, audits, catalogs) would you put in place?** Each department names a data steward accountable for their dataset's quality and access approvals; a central data catalog (Apache Atlas) tracks lineage, ownership, and sensitivity classification for every dataset; a data-sharing review board approves cross-agency access requests; and scheduled access audits verify that granted permissions still match actual need.

2. Proposed Big Data Architecture



3. Tools & Technologies Mapping

Big Data Layer / Function	Tool(s) / Technology Proposed	Why This Tool? (reason)
Data Sources	Census records, economic ministry databases, geospatial/GIS feeds, land/infrastructure registries	Represents the heterogeneous domains that must be integrated into one platform
Ingestion	Apache NiFi / Apache Airflow	Orchestrates scheduled, auditable ETL from legacy department systems with built-in lineage capture and format mapping
Storage	HDFS/S3 data lake (raw/curated/trusted zones) + PostGIS for geospatial data	Zoned storage separates sensitive raw data from vetted, shareable curated/trusted data
Processing (Batch)	Apache Spark / Hive	Standardizes, cleans, and joins large heterogeneous datasets into the canonical schema
Processing (Real-Time)	Kafka-based change data capture (CDC) for incremental updates	Keeps shared datasets reasonably fresh without the cost/complexity of full real-time streaming, which most government use cases don't require
Analytics / ML	Spark MLlib for socio-economic trend analysis, geospatial clustering (DBSCAN)	Supports policy-relevant regional and demographic insight generation for planning
Visualization / Reporting	Power BI / Metabase dashboards, permission-scoped per department	Provides interpretable, access-controlled cross-agency reporting
Security / Governance	Apache Ranger (ABAC policies) + Apache Atlas (catalog/lineage) + encryption at rest	Enforces least-privilege access and full auditability of who accessed what data and when

4. Analytics / Machine Learning Models Used

Analytics here are largely descriptive and inferential rather than predictive, reflecting the platform's planning/policy purpose: regression and correlation analysis link economic indicators (employment, income) to census demographics to support policy targeting; geospatial clustering algorithms (DBSCAN) identify regional patterns such as underserved or high-need areas by combining census and geospatial layers; and a data-quality anomaly-detection model flags inconsistent, duplicate, or out-of-range records arising from department-specific formats before they are promoted from the curated to the trusted zone, protecting downstream analysis from silently propagating bad data.

5. Security, Privacy & Risk Considerations

Personally identifiable fields (names, addresses, national ID numbers) are masked or tokenized in the curated and trusted zones and only decrypted for authorized, fully logged queries approved through the data-sharing review process. Published statistical outputs use differential privacy or k-anonymity thresholds so individuals — particularly in small-population geographic cells — cannot be re-identified from released aggregates. Access follows attribute-based access control (department, role, clearance level, and stated purpose of query) rather than a static role hierarchy, since legitimate access needs vary by context far more than in a typical commercial system. Governance is operationalized through named data stewards per department, mandatory access audits on a recurring schedule, and a formal data-sharing agreement/approval workflow that every cross-agency request must pass through before access is granted — creating both a technical and procedural layer of protection.

6. Key Challenges & Mitigation Strategies

Challenge	Mitigation Strategy
Departments resist sharing data due to legacy system incompatibility or turf concerns	Start integration with a small set of high-value shared entities (e.g., a common address/geocode registry) to demonstrate value before expanding scope, and formalize data-sharing agreements with clear mutual benefit
Re-identification risk when combining census, economic,	Apply k-anonymity/differential privacy at the point of aggregation, and suppress or coarsen geographic cells below a minimum population threshold

Challenge	Mitigation Strategy
and geospatial data at fine granularity	
Maintaining data quality and consistency across many independently-managed source systems	Enforce schema validation and data-quality checks at the curated-zone boundary, with named data stewards accountable for their department's data quality
Keeping the access-control policy manageable as departments and use cases grow	Use ABAC with centrally managed policy definitions (Apache Ranger) rather than per-application access logic, so policy changes propagate consistently everywhere

7. Conclusion / Business Value

A zoned, catalog-governed platform lets departments integrate census, economic, and geospatial data for shared policy insight — supporting better-targeted public services and infrastructure planning — while enforcing strict, auditable, attribute-based access controls. This balances the platform's core objective (cross-agency interoperability) against its equally critical constraint (citizen privacy and regulatory compliance), rather than treating governance as an afterthought bolted onto a purely technical integration layer.

Problem 4: Banking Fraud Detection

Scenario

A bank processes millions of financial transactions every minute. It needs to detect fraudulent activities in real time. The system must analyze historical and streaming transaction data.

Low latency and high accuracy are critical requirements. Design a big data pipeline for fraud detection. Explain the technologies and algorithms you would implement.

Focus Areas for This Problem

- Millions of transactions per minute with strict low-latency requirements
- Combining historical patterns with live streaming data
- High accuracy fraud scoring with minimal false alarms

1. Problem Understanding & Industry Context

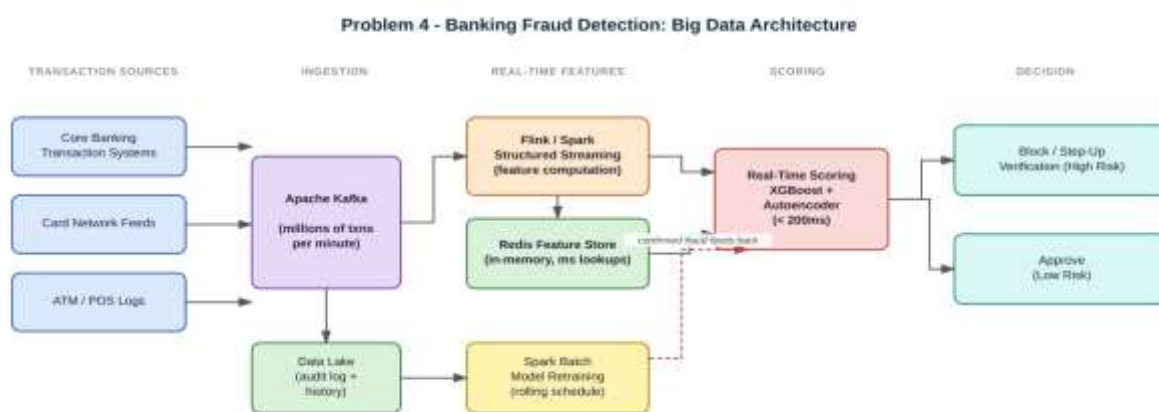
Banking fraud detection is one of the most latency-sensitive big data use cases in industry: a scoring decision must return in well under 100-200 milliseconds so a transaction can be approved or blocked at the point of authorization, while still drawing on months of historical behavioral context to judge whether a given transaction is normal for that specific customer. This requires a strict architectural separation between a real-time scoring path (stateless, millisecond-level feature lookups against a pre-computed feature store) and a batch/offline path that periodically retrains models on the full historical labeled dataset — attempting to do both in one system would either blow the latency budget or starve the model of the historical context it needs for accuracy. The financial stakes also mean the system must minimize two very different kinds of error simultaneously: missed fraud (direct financial loss and regulatory exposure) and false declines (customer friction and lost trust), which shapes both the model choice and the operational workflow around it.

Guiding Questions — Answered

- **What latency target would you design for, and which technologies help you hit it?** A target of under 100-200 milliseconds end-to-end, since the score must return before the card network's authorization timeout. This is achieved with Kafka for ordered low-latency ingestion, Flink/Spark Structured Streaming for real-time feature computation, and a Redis-backed feature store so feature lookups are in-memory rather than round-tripping to a disk-based database.

- **Which machine learning algorithm(s) suit real-time fraud scoring, and why?** XGBoost (or a Random Forest) for the primary supervised score, because gradient-boosted trees are fast at inference, handle mixed tabular features well, and are reasonably explainable via feature importance/SHAP values — useful for regulatory justification of declines. An autoencoder is added for unsupervised anomaly detection of fraud patterns not yet present in labeled training data.
- **How would historical transaction data be used to train or improve the model?** Millions of historical labeled transactions (confirmed fraud vs. legitimate) train the supervised model in a nightly/weekly Spark batch job; confirmed fraud cases from ongoing operations are continuously fed back into the training set so the model adapts to evolving fraud tactics rather than staying static.
- **How would you ensure the system stays available and accurate under peak load?** Horizontally scale the Kafka partitions and Flink/Spark Streaming task parallelism based on throughput, keep the feature store in a distributed in-memory cache (Redis Cluster) that scales independently, and continuously monitor model accuracy/latency in production so degradation under load is caught immediately rather than discovered after losses occur.

2. Proposed Big Data Architecture



3. Tools & Technologies Mapping

Big Data Layer / Function	Tool(s) / Technology Proposed	Why This Tool? (reason)
Data Sources	Core banking transaction systems, card network feeds, ATM/POS logs	Captures both the transaction event and its channel/context, both needed for accurate scoring
Ingestion	Apache Kafka	Ordered, high-throughput ingestion capable of handling millions of transactions per minute with strict ordering per account
Storage	HDFS/S3 data lake for history + HBase/Cassandra + Redis for low-latency feature lookups	Separates cheap, durable historical storage from the millisecond-latency serving storage the real-time path needs
Processing (Batch)	Apache Spark	Retrains fraud models on the full historical labeled dataset and validates performance before redeployment
Processing (Real-Time)	Apache Flink / Spark Structured Streaming	Computes real-time behavioral features and triggers sub-second fraud scoring per transaction
Analytics / ML	XGBoost classifier + autoencoder anomaly detection	Combines supervised accuracy on known fraud patterns with unsupervised detection of novel, previously unseen fraud
Visualization / Reporting	Grafana dashboards for the fraud operations team	Real-time monitoring of flagged transactions, model latency, and system health under load

Big Data Layer / Function	Tool(s) / Technology Proposed	Why This Tool? (reason)
Security / Governance	Tokenization, PCI-DSS controls, encryption at rest/in transit	Protects card and account data end-to-end and satisfies regulatory compliance obligations

4. Analytics / Machine Learning Models Used

Primary supervised model: an XGBoost (or Random Forest) classifier trained on historical labeled transactions (fraud vs. legitimate) provides the real-time score, using engineered features such as transaction velocity over rolling windows, amount deviation from the customer's normal spend pattern, merchant risk category, and geo-distance from the customer's last known transaction location. Gradient-boosted trees are chosen for their strong tabular performance, fast millisecond-level inference, and reasonable explainability via SHAP feature-importance values, which supports regulatory requirements to justify declined transactions.

Unsupervised anomaly detection: an autoencoder trained only on legitimate transactions learns to reconstruct normal spending patterns; transactions with high reconstruction error are flagged as potential fraud even if they don't match any previously labeled fraud pattern, catching novel fraud tactics that a purely supervised model would miss.

Continuous improvement: both models are retrained on a rolling schedule using newly confirmed fraud cases fed back from investigation outcomes, and their live performance (precision, recall, false-positive rate) is tracked so any drift is caught quickly rather than allowed to silently erode accuracy.

5. Security, Privacy & Risk Considerations

The pipeline must be fully PCI-DSS compliant: card numbers are tokenized at capture and never stored or processed in raw form within the analytics environment, and all data is encrypted both at rest and in transit. Access to raw transaction data is tightly role-restricted, with every access logged for regulatory audit trails. Because fraud decisions affect customers directly and are subject to regulatory scrutiny, the model must be explainable enough to produce reason codes for declined transactions on request, and model performance/drift is continuously monitored in production so accuracy doesn't silently degrade —

whether from peak-load stress, changing fraud patterns, or data-quality issues upstream. Disaster-recovery and failover planning for the real-time scoring service is also essential, since any downtime directly blocks legitimate transactions.

6. Key Challenges & Mitigation Strategies

Challenge	Mitigation Strategy
Meeting sub-200ms latency at millions of transactions per minute	Keep the real-time path stateless and feature lookups in-memory (Redis), and horizontally scale Kafka partitions and stream-processing parallelism ahead of peak load
Minimizing false declines that frustrate legitimate customers	Use a tiered response (step-up verification for medium-risk instead of outright decline) and continuously tune the classification threshold against live precision/recall metrics
Detecting novel fraud patterns not present in historical labeled data	Layer an unsupervised autoencoder alongside the supervised classifier so new fraud tactics are still flagged via anomaly detection
Regulatory requirement to explain why a transaction was declined	Use tree-based models with SHAP value explainability rather than a black-box deep model for the primary decisioning score

7. Conclusion / Business Value

This pipeline delivers sub-second fraud scoring at millions-of-transactions-per-minute scale by cleanly separating a lightweight real-time scoring path from a heavier offline retraining loop. The combination of a fast, explainable supervised model and an unsupervised anomaly detector reduces financial losses from both known and novel fraud patterns, while tiered decisioning and continuous performance monitoring keep false declines — and the resulting customer friction — low, satisfying both the business and regulatory requirements of the use case.

Problem 5: Agriculture Smart Farming

Scenario

A smart farming system collects soil, weather, and crop data. Sensors and drones provide real-time field information. Farmers want insights to improve yield and resource usage. Data is generated from multiple distributed sources. Design a big data solution for precision agriculture. Explain how analytics can support decision-making.

Focus Areas for This Problem

- Distributed, often low-connectivity data sources (sensors, drones)
- Combining soil, weather, and crop data for actionable insight
- Supporting farmer decision-making with clear, timely analytics

1. Problem Understanding & Industry Context

Precision agriculture is unique among these five scenarios because connectivity, not data volume, is the primary engineering constraint: field sensors and drones typically operate in remote rural areas with intermittent or no network access, so the system must be designed around store-and-forward data collection rather than assuming continuous streaming like the retail or banking scenarios. Combining soil, weather, and crop-imagery data is valuable precisely because no single data type tells the full story — soil moisture alone doesn't indicate whether a field actually needs irrigation without also knowing the weather forecast and current crop growth stage, and drone imagery adds an early-warning signal for stress/disease that ground sensors cannot capture. Because the end users are farmers rather than data engineers or analysts, the entire value chain must terminate in a simple, low-bandwidth, actionable recommendation rather than a raw analytics dashboard — insight that never reaches or is understood by the farmer delivers zero value regardless of how sophisticated the underlying models are.

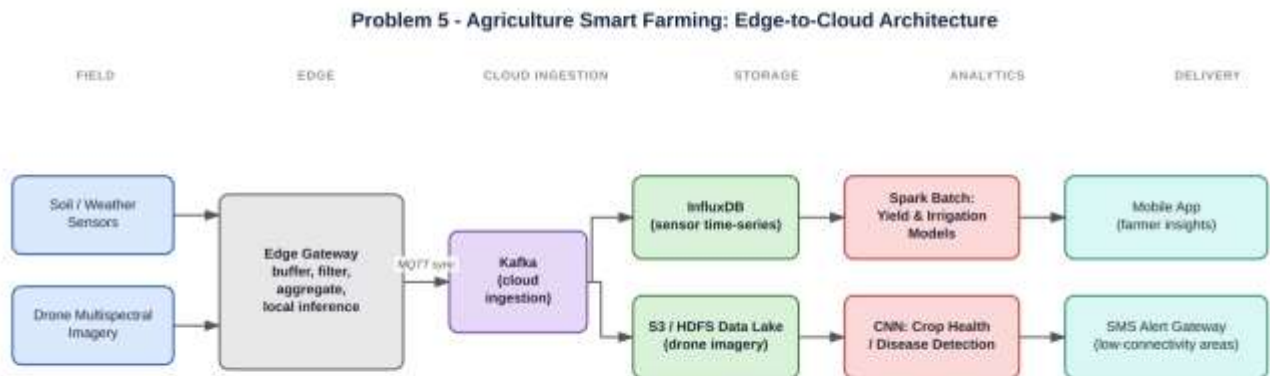
Guiding Questions — Answered

- **How would you handle intermittent connectivity from remote field sensors?** Sensors report to a local edge gateway that buffers readings on local storage; the gateway syncs to the cloud opportunistically over MQTT whenever connectivity becomes available, using a store-and-forward pattern so no data is lost during outages, only delayed.
- **What edge processing (if any) would you do before sending data to the cloud?** The edge gateway aggregates raw per-second readings into hourly averages/summaries to conserve

bandwidth, applies basic filtering to drop obviously faulty sensor readings, and can run a lightweight on-device model for immediate local alerts (e.g., critically low soil moisture) that don't need to wait for a cloud round-trip.

- **What predictive models would help farmers with irrigation or yield decisions?** A regression/random-forest model combining soil moisture, weather forecast, and crop growth stage recommends irrigation timing and volume; a CNN analyzing drone multispectral imagery detects early crop stress or disease; and a time-series yield-prediction model trained on historical seasonal data estimates expected harvest for planning.
- **How would insights be delivered to farmers in a usable way (app, SMS, dashboard)?** A lightweight mobile app for farmers with smartphone/data access, and SMS-based alerts for those in areas with poor connectivity or lower digital literacy, so the same underlying insight (e.g., 'irrigate field 3 today') reaches farmers through whichever channel actually works for them.

2. Proposed Big Data Architecture



3. Tools & Technologies Mapping

Big Data Layer / Function	Tool(s) / Technology Proposed	Why This Tool? (reason)
Data Sources	Soil moisture/temperature sensors, weather stations, drone multispectral imagery	Covers the three complementary data types needed for a full picture of field conditions
Ingestion	MQTT (edge sync) + Apache Kafka (cloud ingestion)	MQTT is lightweight and purpose-built for intermittent-connectivity IoT devices; Kafka scales cloud-side ingestion once data arrives
Storage	InfluxDB (time-series sensor data) + S3/HDFS data lake (drone imagery)	Matches storage engine to data shape — time-series database for regular sensor readings, object storage for large imagery files
Processing (Batch)	Apache Spark	Runs yield-prediction and irrigation-optimization models over accumulated seasonal data
Processing (Real-Time)	Edge-device inference on the gateway + opportunistic streaming sync	Enables immediate local alerts (e.g., very low soil moisture) even without cloud connectivity
Analytics / ML	CNN for crop health/disease detection from drone imagery; regression/random forest for yield prediction and irrigation recommendations	Matches model type to data type — a vision model for imagery, tabular models for structured sensor/weather data

Big Data Layer / Function	Tool(s) / Technology Proposed	Why This Tool? (reason)
Visualization / Reporting	Mobile app + SMS alert gateway	Reaches farmers with limited smartphone or data access in a simple, usable, low-bandwidth way
Security / Governance	Device authentication + TLS-encrypted MQTT, farmer data-ownership policy	Prevents spoofed sensor readings and clarifies who owns and can use the collected farm data

4. Analytics / Machine Learning Models Used

Crop health monitoring: a convolutional neural network (CNN) analyzes drone multispectral imagery (e.g., NDVI-derived bands) to detect crop stress, disease, or pest damage earlier than visual inspection would catch it, flagging specific field zones for the farmer to inspect.

Irrigation recommendation: a regression or random-forest model combines current soil moisture, the weather forecast, and the crop's current growth stage to recommend irrigation timing and volume, reducing water waste compared to fixed-schedule irrigation.

Yield prediction: a time-series model trained on historical seasonal data (soil, weather, and past yield outcomes) estimates the expected harvest ahead of time, helping farmers plan storage, labor, and sales contracts with more confidence than experience-based estimation alone.

5. Security, Privacy & Risk Considerations

Field devices must authenticate before publishing data, preventing spoofed sensor readings that could otherwise mislead irrigation or yield decisions with real financial consequences for the farmer. MQTT traffic is TLS-encrypted despite the added overhead, since data integrity matters more than raw bandwidth savings in this low-trust, physically exposed device environment. Because farm data can reveal commercially sensitive information (expected yield, land use patterns), the platform should adopt a clear data-ownership policy stating that farmers — not the platform operator — own their raw field data, with explicit opt-in consent required for any aggregated benchmarking or data-sharing with third parties such as insurers or input suppliers.

6. Key Challenges & Mitigation Strategies

Challenge	Mitigation Strategy
Data loss or staleness from intermittent field connectivity	Use store-and-forward buffering at the edge gateway so readings are queued locally and synced once connectivity returns, rather than dropped
Limited bandwidth for transmitting high-resolution drone imagery	Perform initial image analysis/compression at the edge where possible, and only transmit flagged regions of interest at full resolution
Farmers with low digital literacy or smartphone access not receiving insights	Deliver the same recommendation through multiple channels (app and SMS) so the insight reaches the farmer through whichever channel they actually use
Sensor drift or faulty hardware producing misleading readings over time	Apply basic anomaly filtering at the edge and periodic sensor calibration/validation against known reference readings

7. Conclusion / Business Value

By pushing lightweight processing to the edge and syncing opportunistically, this design works within the real connectivity constraints of rural fields rather than assuming reliable networking it won't have. CNN-based imagery analysis, irrigation-recommendation models, and yield prediction turn raw, scattered sensor and drone data into concrete, SMS/app-deliverable recommendations — helping farmers use water and inputs more efficiently, catch crop problems earlier, and plan around a more accurate yield estimate, which directly improves both productivity and profitability.